

# Personal Loan Acceptance Prediction Using Machine Learning: A Binary Classification Approach

## 1. INTRODUCTION

Commercial banking relies heavily on converting liability customers (depositors) into asset customers (borrowers). This project builds a classification model to predict whether a retail bank customer will accept a personal loan offer based on their demographic and financial profile. The target variable is Personal Loan, framing this as a binary classification problem (1 = accepted, 0 = declined). Accurately identifying high-propensity borrowers is a critical operational challenge in retail banking. The motivation for this project is to apply machine learning to the loan targeting process, bridging the gap between raw financial data and actionable marketing strategies to optimize marketing conversion rates. The dataset was obtained from Kaggle (<https://www.kaggle.com/code/alirezachahardoli/bank-personal-loan-modelling>). It is a commercial banking dataset containing 5,000 records of retail customer data, capturing demographic information, financial metrics, and indicators of existing bank relationships.

## 2. EXPLORATORY DATA ANALYSIS Dataset Overview & Missing Values

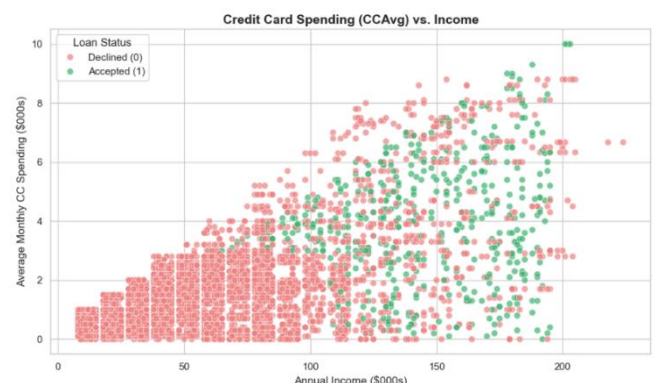
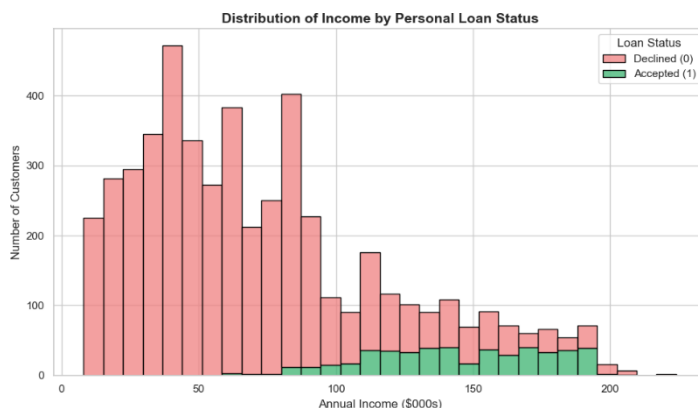
The dataset contains 5000 records and 14 features after removing ID and ZIP Code. Each row represents a single retail bank customer. Features include:

- Age — customer age in completed years
- Experience — years of professional experience
- Income — annual income (\$000s)
- Family — family size (1–4)
- CCAvg — average monthly credit card spending (\$000s)
- Education — level: 1=Undergrad, 2=Graduate, 3=Professional
- Mortgage — value of home mortgage, if any (\$000s)
- Securities Account — binary: holds a securities account with the bank
- CD Account — binary: holds a certificate of deposit account
- Online — binary: uses internet banking
- CreditCard — binary: uses a Universal Bank credit card
- Personal Loan — target: accepted loan offer (1) or not (0)

There were exactly 0 missing values across all features, so no imputation was required. The target variable, Personal Loan, is highly imbalanced approximately 90.4% of customers declined the loan, and only 9.6% accepted. Because accuracy alone is misleading on imbalanced data, the F1-Score was chosen as the primary evaluation metric to properly balance precision and recall for the minority class.

### Key EDA Insight

The income distribution reveals a strong signal loan acceptors are heavily concentrated in the \$100,000 to \$200,000 annual income range, while most decliners earn under \$100,000. This suggests income acts as a prerequisite threshold for loan eligibility. Another key insight is that Accepted customers cluster in the high-income/high-spending quadrant, while declined customers are densely packed at lower income and spending levels. High earners who spend heavily on credit are disproportionately likely to accept the loan.



### 3. DATA PREPARATION AND PREPROCESSING Data Cleaning & Feature Engineering

The ID and ZIP Code columns were dropped as they identify geographic variables with no generalized predictive value. The Education column was re-mapped from integer codes (1, 2, 3) to descriptive labels (Undergrad, Graduate, Professional) prior to encoding to ensure interpretable feature names. Because this is a classification task rather than regression, no logarithmic transformation of the target variable was applicable.

**Train/Validation/Test Split & Imbalance Handling:** While synthetic data generation techniques like SMOTE were considered to balance the dataset, they were ultimately excluded to completely avoid data leakage. Instead, the imbalance was handled naturally by splitting the data into three stratified subsets: 60% training, 20% validation, and 20% test. Stratified splitting ensured the 90/10 class ratio was preserved across all sets. This single split was used for all model fitting.

#### Preprocessing Pipeline

A unified ColumnTransformer preprocessing pipeline was built to prevent data leakage and handle feature scaling:

- **Numeric features** (Age, Experience, Income, CCAvg, Family) were standardized using StandardScaler.
- **Categorical features** (Education) were one-hot encoded with drop='first' to avoid multicollinearity.
- **Binary features** (Securities Account, CD Account, Online, CreditCard) were passed through unchanged.

### 4. MODEL TRAINING AND RESULTS

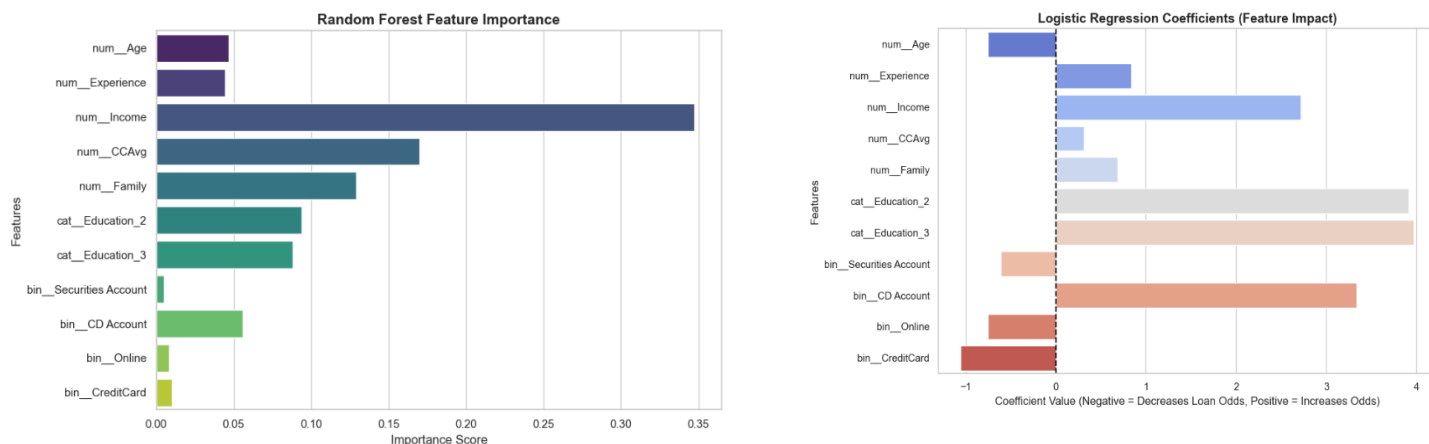
| Model Name          | Hyperparameters Tried                                                          | Best Hyperparameters                                                                  | Mean CV F1 Score | Std Dev F1 Score | Test F1 Score |
|---------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|------------------|------------------|---------------|
| Logistic Regression | C: [0.01, 0.1, 1, 10, 100]                                                     | {'classifier__C': 10}                                                                 | 0.7565           | 0.0337           | 0.7602        |
| Decision Tree       | max_depth: [3, 5, 10, 15, None],<br>min_samples_ min_samples_split: [2, 5, 10] | {'classifier__max_depth': 10,<br>'classifier__min_samples_split': 2}                  | 0.922            | 0.0248           | 0.934         |
| SVM                 | C: [0.1, 1, 10], kernel: ['linear', 'rbf'],<br>gamma: ['scale', 'auto']        | {'classifier__C': 10, 'classifier__gamma': 'scale', 'classifier__kernel': 'rbf'}      | 0.9205           | 0.0204           | 0.9198        |
| Random Forest       | n_estimators: [50, 100, 150],<br>max_depth: [5, 10, None]                      | {'classifier__max_depth': None, 'classifier__scale', 'classifier__n_estimators': 100} | 0.9334           | 0.0221           | 0.9684        |
| Gradient Boosting   | Early Stopping (val_fraction=0.2, patience=10)                                 | Early Stopping Reached                                                                | Val F1: 0.9032   | N/A              | 0.9588        |
| Neural Network      | Configs: 16->1; 32->Drop->16->1; 64->Drop->32->Drop->16->1                     | Config 1 (Simple)                                                                     | Val F1: 0.8962   | N/A              | 0.9319        |

Six models representing different machine learning algorithms were evaluated. Grid Search (with cross-validation) was performed to tune hyperparameters for Logistic Regression, Decision Tree, SVM, and Random Forest. Per requirements, Gradient Boosting and the Neural Network bypassed Grid Search; Gradient Boosting utilized Early Stopping (evaluated against a 20% validation set with a patience of 10). The Neural Network was tuned by comparing three distinct architectures (16→1; 32→Dropout→16→1;

64→Dropout→32→Dropout→16→1). Ultimately, Random Forest achieved the highest overall performance with a Test F1 Score of 0.9684, while the Neural Network's best performance (Test F1 Score: 0.9319) was achieved using the simplest architecture.

### Additional Results (Feature Impact)

Random Forest feature importance confirmed that Income, CCAvg, and Education\_Professional are the strongest drivers of the ensemble's predictions. Logistic Regression coefficient analysis revealed that Income and Education (Professional level) carry the largest positive coefficients, substantially increasing the odds of loan acceptance, while holding a CD Account also contributes positively.



## 5. CONCLUSIONS AND NEXT STEPS Best Model

The Random Forest classifier emerged as the best overall model, achieving the highest Test F1-Score (0.9684). Its ensemble approach effectively captured the non-linear relationships in this dataset. Gradient Boosting (with early stopping) was a strong runner-up, achieving a highly competitive Test F1-Score of 0.9588. While Random Forest performed best overall, SVM demonstrated the highest stability across cross-validation folds with the lowest standard deviation (0.0204). Finally, while Logistic Regression was the weakest performer overall, it remains the simplest model and offers fully readable coefficients if pure interpretability is required by business stakeholders.

### Next Steps

To further improve model performance, testing SMOTE oversampling via an imblearn pipeline would directly address the 90/10 class imbalance by comparing balanced versus unbalanced F1 results across all models. Additionally, engineering interaction features such as an  $\text{Income} \times \text{CCAvg}$  metric or a Mortgage-to-Income ratio could expose financial patterns not captured by the raw features alone, given that the EDA already revealed a strong joint relationship between income and credit spending. A full hyperparameter grid search on Gradient Boosting, tuning `learning_rate`, `max_depth`, and `subsample` that would provide more rigorous optimization than early stopping alone and could potentially push it past Random Forest's leading test F1 of 0.9684. Finally, exploring XGBoost or LightGBM as alternative boosting frameworks would be a natural next step, as both handle class imbalance natively and consistently outperform scikit-learn's Gradient Boosting on tabular datasets.